JSON Refiner Advanced Edition

TEAM LEADER:

A.INDUJA

TEAM MEMBER :

M.KALAIMATHI

K.VARSHINI PRIYA

B.VANITHA

JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

Project Description

JSON Refiner Advanced Edition is an intelligent JSON processing platform that leverages Python's advanced data handling capabilities to provide comprehensive JSON parsing, validation, transformation, and refinement with professional styling through Gradio's interactive interface. The platform addresses the challenge of JSON data quality and format standardization by delivering AI-powered JSON generation, automatic type inference, schema validation, case conversion, flattening/unflattening, and production-ready output.

Using sophisticated parsing algorithms and Python's type system, the system analyzes unstructured key-value data, automatically infers correct data types (strings, numbers, booleans, nulls, arrays, objects), applies consistent naming conventions (snake_case, camelCase, kebab-case, PascalCase), validates against JSON schemas, and provides multiple transformation options. The system ensures professional output through Gradio's user-friendly interface while maintaining data integrity through comprehensive validation and error reporting.

JSON Refiner Advanced transforms JSON processing into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and advanced algorithms that handle complex transformations while maintaining data consistency, accessibility standards, and production-ready code quality.

Scenarios

Scenario 1: Data Migration from CSV to JSON

A data analyst needs to convert a flat CSV file with mixed data types into structured JSON format. The system receives key-value pairs from the CSV (name: Vaishali, age: 20, active: true, score: 95.5), automatically detects the correct data types, applies consistent naming conventions, and generates clean, production-ready JSON. The analyst can validate against a predefined schema to ensure data quality before migration to their database system.

Scenario 2: API Response Normalization

A backend developer receives JSON responses from multiple third-party APIs with inconsistent key naming conventions and data structures. The system standardizes all keys to a consistent case style (e.g., converting FirstName, first_name, firstName to a single convention), merges multiple API responses into a unified structure, removes null/empty values, validates against a master schema, and produces clean data for frontend consumption. This eliminates the need for manual data transformation in application logic.

Scenario 3: Configuration File Management

A DevOps engineer manages configuration files in simple text format and needs to convert them to JSON for automated deployment systems. The system parses the configuration (database_host: localhost, port: 5432, ssl_enabled: true, retry_count: 3), infers types correctly, validates required fields, prevents duplicate keys with detailed error reporting, and generates valid JSON ready for infrastructure-as-code tools. The engineer can maintain human-readable configs while ensuring JSON compliance.

Scenario 4: Data Flattening for Analytics

An analytics team has deeply nested JSON structures from their application and needs to flatten them for spreadsheet analysis. The system converts nested objects like user: {name: "Vaishali", profile: {age: 20}} to flat dot-notation (user.name: "Vaishali", user.profile.age: 20), making the data compatible with CSV/Excel tools. Later, they can unflatten the data back to nested structure when needed, maintaining complete reversibility.

Scenario 5: Schema-Based Data Validation

A quality assurance team validates JSON data against company standards using JSON Schema. The system accepts raw data, applies transformations, validates

structure and field requirements, provides detailed error messages for missing/invalid fields, and prevents invalid data from entering the pipeline. The comprehensive validation ensures data consistency across all systems and catches errors early.

Architecture Overview

JSON Refiner Advanced is built as a modular platform combining Python backend with sophisticated JSON processing algorithms for intelligent data transformation. The architecture prioritizes data integrity, user experience, and production readiness through Gradio's interactive interface and comprehensive validation mechanisms.

Core Technologies

- Python 3.8+: Core programming language for robust data processing
- Gradio 4.0+: Interactive web interface framework for user interaction
- JSON5: Advanced JSON parsing for flexible input handling
- JSONSchema: Schema validation and data conformance checking
- Regular Expressions: Pattern matching for advanced parsing
- Google Colab: Cloud-based development and deployment environment
- HTML5/CSS3: Modern web standards for interface rendering

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- pip: Python package manager (comes with Python)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and Gradio sharing

Knowledge Prerequisites

- Python Basics: Functions, dictionaries, list comprehensions, string manipulation
- JSON Fundamentals: JSON structure, types, and validation
- Regular Expressions: Pattern matching for data parsing
- Gradio Framework: Understanding of interface components and callbacks
- Data Validation: Familiarity with JSON Schema and validation concepts
- Google Colab: Basic notebook interface and cell execution

Hardware Requirements

- Processor: Any modern processor (Intel i3 or equivalent)
- RAM: Minimum 2GB local (Colab provides cloud resources)
- Storage: 500MB free space for application and generated files
- Display: Minimum 1024x768 resolution for interface usability

Project Workflow

Phase 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies & Configure Environment

- Create requirements.txt with all dependencies
- Set up Google Colab environment with proper settings
- Install required packages: gradio, json5, jsonschema, python-dotenv, pydantic
- Verify all packages are installed correctly

Activity 1.2: Design Core Data Structures

- Define CaseStyle Enum for case conversion options
- Create JSONRefinerCore class with all processing methods
- Design type inference system for automatic data type detection
- Implement error tracking and logging mechanisms

Phase 2: Core JSON Processing Engine

Activity 2.1: Implement Type Inference System

- Create infer_type() method for automatic type detection
- Handle boolean conversions (true/false, yes/no, on/off)
- Implement null value handling (null, none, nil, n/a)
- Parse numbers (integers and floats)
- Process arrays and objects from JSON strings
- Default to string for unrecognized values

Activity 2.2: Build Key Normalization & Case Conversion

- Implement to_snake_case() conversion
- Implement to_camelCase() conversion
- Implement to_kebab_case() conversion
- Implement to_pascal_case() conversion
- Create normalize_key() method that applies selected case style

- Test all case conversion combinations

Phase 3: Validation & Transformation Features

Activity 3.1: Implement JSON Schema Validation

- Create validate_json_schema() method
- Use jsonschema library for Draft7 validation
- Implement error message formatting
- Test against various schema definitions

Activity 3.2: Build Required Fields Checking

- Create check_required_fields() method
- Implement field presence validation
- Generate missing field reports
- Combine with schema validation results

Activity 3.3: Develop JSON Transformation Methods

- Implement flatten_json() for nested to flat conversion
- Implement unflatten_json() for dot-notation reversal
- Create merge_json_objects() for deep merging
- Implement remove_null_values() for data cleanup
- Test all transformation methods with complex data

Phase 4: Gradio Interface Development

Activity 4.1: Design Core Refining Tab

- Create input fields for key-value text
- Add case style radio button selector
- Implement remove nulls and flatten checkboxes
- Build JSON output code display
- Add status and statistics output panels

Phase 5: Integration, Testing & Deployment

Activity 5.1: Google Colab Deployment

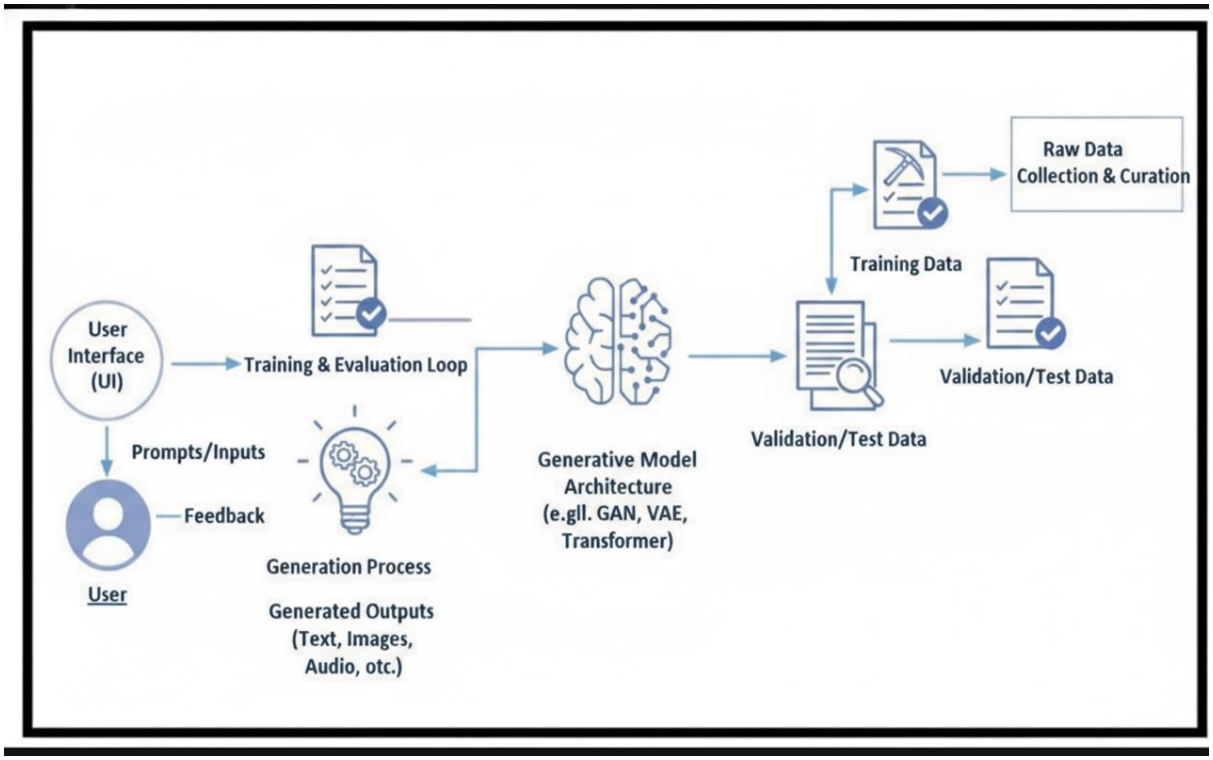
- Connect all interface tabs
- Test data flow between components

- Verify transformation pipeline
- Test error handling

Activity 5.2: Comprehensive Testing

- Test all case conversion styles
- Validate type inference accuracy
- Test schema validation with various schemas
- Test merging with conflicting keys
- Test flattening with deeply nested objects

Technical Architecture



MILESTONE 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies

Load the model :

Create requirements.txt with the following dependencies:

- gradio==4.0+
- json5==0.9.6+
- jsonschema==4.17.0+
- python-dotenv==0.19.0+
- pydantic==1.9.0+

```
!pip install -q gradio json5 jsonschema python-dotenv pydantic

import gradio as gr
import json
import re
import ast
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Any, Tuple, Optional
import json5
from jsonschema import validate, ValidationError, Draft7Validator
from enum import Enum

# ==============================================================================
# JSON REFINER - ADVANCED VERSION WITH PREMIUM FEATURES (FIXED)
# ==============================================================================
```

Activity 1.2: Initialize Enum and Core Class

Create CaseStyle Enum:

```
class CaseStyle(Enum):
        SNAKE_CASE = "snake_case"
        CAMEL_CASE = "camelCase"
        KEBAB_CASE = "kebab-case"
        PASCAL_CASE = "PascalCase"
```

Initialize JSONRefinerCore class with tracking:

```
class JSONRefinerCore:
        def __init__(self):
        self.conversion_log = []
        self.validation_errors = []
```

self.transformation_history = []

```python
# ==============================================================================
# JSON REFINER - ADVANCED VERSION WITH PREMIUM FEATURES (FIXED)
# ==============================================================================

class CaseStyle(Enum):
    """Case conversion styles"""
    SNAKE_CASE = "snake_case"
    CAMEL_CASE = "camelCase"
    KEBAB_CASE = "kebab-case"
    PASCAL_CASE = "PascalCase"

class JSONRefinerCore:
    """Advanced JSON parsing and refinement engine"""

    def __init__(self):
        self.conversion_log = []
        self.validation_errors = []
        self.transformation_history = []
```

MILESTONE 2: Type Inference & Case Conversion System

Activity 2.1: Implement Type Inference

The type inference system automatically detects and converts values:

- Booleans: Recognizes true, false, yes, no, on, off, 1, 0
- Nulls: Handles null, none, nil, undefined, n/a
- Numbers: Detects integers and floating-point numbers
- Arrays: Parses JSON arrays from string format
- Objects: Parses JSON objects from string format
- Strings: Default fallback for unrecognized values

Example: "age: 20" → {"age": 20} (integer)
Example: "active: true" → {"active": true} (boolean)

```python
            return JSONRefinerCore.to_pascal_case(key)
        return key

    @staticmethod
    def infer_type(value: str) -> Any:
        """Infer and convert value to correct JSON type"""
        value = value.strip()

        if value.lower() in ['null', 'none', 'nil', 'undefined', 'n/a', 'na']:
            return None

        if value.lower() in ['true', 'yes', 'on', '1']:
            return True
        if value.lower() in ['false', 'no', 'off', '0']:
            return False

        try:
            if '.' not in value:
                return int(value)
        except ValueError:
            pass
```

Activity 2.2: Build Case Conversion Methods

Implement four case conversion standards:

1. snake_case: user_name, first_name, is_active
2. camelCase: userName, firstName, isActive
3. kebab-case: user-name, first-name, is-active
4. PascalCase: UserName, FirstName, IsActive

Each method uses regex patterns to normalize input:
- Remove non-alphanumeric characters
- Split on boundaries (spaces, underscores, hyphens)
- Apply case transformation
- Validate output format

```python
def convert_case(json_text: str, target_case: str) -> Tuple[str, str]:
    """Convert JSON keys to different case styles"""
    try:
        data = json.loads(json_text)
        case_enum = CaseStyle[target_case.upper().replace('-', '_')]

        def convert_keys(obj):
            if isinstance(obj, dict):
                return {
                    refiner.normalize_key(k, case_enum): convert_keys(v)
                    for k, v in obj.items()
                }
            elif isinstance(obj, list):
                return [convert_keys(item) for item in obj]
            return obj

        converted = convert_keys(data)
        output = json.dumps(converted, indent=2, ensure_ascii=False)
        return output, f"✅ Converted to {target_case}!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"
```

MILESTONE 3: Validation & Transformation Features

Activity 3.1: JSON Parsing & Validation

Key-value parsing system:

```python
    @staticmethod
    def parse_key_value_text(text: str, case_style: CaseStyle = CaseStyle.SNAKE_CASE) -> Tuple[Dict, List[str]]:
        """Parse key-value text into dictionary"""
        result = {}
        errors = []
        duplicates = []

        lines = text.split('\n')

        for line_num, line in enumerate(lines, 1):
            line = line.strip()

            if not line or line.startswith('#'):
                continue

            if ':' not in line:
                errors.append(f"Line {line_num}: No colon separator found - '{line}'")
                continue

            try:
                key, value = line.split(':', 1)
                key = key.strip()
```

Activity 3.2: Schema & Field Validation

Dual validation approach:

1. Required Fields: Check for mandatory fields
   - Input: "name, age, email"

- Result: Lists missing fields

2. Schema Validation: Validate against JSON Schema
   - Input: {"type": "object", "properties": {...}}
   - Result: Full structural validation
   - Supports Draft 7 standard

```python
@staticmethod
def validate_json_schema(data: Dict, schema: Dict) -> Tuple[bool, List[str]]:
    """Validate JSON against schema"""
    errors = []
    try:
        validate(instance=data, schema=schema)
        return True, []
    except ValidationError as e:
        return False, [str(e)]
    except Exception as e:
        return False, [f"Schema validation error: {str(e)}"]

@staticmethod
def check_required_fields(data: Dict, required_fields: List[str]) -> Tuple[bool, List[str]]:
    """Check for required fields"""
    errors = []
    missing = []
    for field in required_fields:
        if field not in data:
            missing.append(field)

    if missing:
        return False, [f"Missing required fields: {', '.join(missing)}"]
    return True, []
```

Activity 3.3: JSON Transformation Operations

Multiple transformation capabilities:

1. Flatten: Convert nested objects to dot-notation
   Input: {"user": {"name": "Vaishali", "age": 20}}
   Output: {"user.name": "Vaishali", "user.age": 20}

2. Unflatten: Reverse flattening
   Input: {"user.name": "Vaishali", "user.age": 20}
   Output: {"user": {"name": "Vaishali", "age": 20}}

3. Merge: Deep merge multiple JSON objects
4. Clean: Remove null/undefined values
5. Stats: Generate detailed statistics

MILESTONE 4: Gradio Interface Development

Activity 4.1: focuses on designing and implementing a comprehensive multi-functional JSON refinement interface that enables users to validate, transform, analyze, and manage JSON data efficiently through an intuitive tab-based UI.

The Gradio interface includes 7 functional tabs:

1. Core Refining: Convert key-value text to JSON

- Input: Text with key-value pairs
- Options: Case style, remove nulls, flatten
- Output: JSON code, status, statistics

```python
# ============= TAB 1: Core Refining =============
with gr.TabItem("🔧 Core Refining"):
    with gr.Row():
        with gr.Column(scale=1):
            gr.Markdown("### 📝 Input Format")
            text_input = gr.Textbox(
                label="Key-Value Text",
                lines=12,
                value="name: Vaishali David\nage: 20\nis_active: true\nbalance: 1500.50\nemail: Vaishali@gmail.com\ntags: [\"admin\", \"user\"]
            )

            case_style = gr.Radio(
                ["snake_case", "camelCase", "kebab-case", "PascalCase"],
                value="snake_case",
                label="🎨 Key Case Style"
            )

            with gr.Row():
                remove_nulls = gr.Checkbox(label="Remove Nulls", value=False)
                flatten = gr.Checkbox(label="Flatten", value=False)
                pretty_print = gr.Checkbox(label="Pretty Print", value=True)

            refine_btn = gr.Button("✨ Refine JSON", size="lg", variant="primary")
```

2. Validation: Validate JSON structure
   - Input: JSON, schema, required fields
   - Output: Validation results

```python
# ============= TAB 2: Validation =============
with gr.TabItem("✅ Validation"):
    gr.Markdown("### 🔍 Validate JSON Structure")

    with gr.Row():
        with gr.Column(scale=1):
            json_to_validate = gr.Code(
                language="json",
                label="JSON to Validate",
                lines=10,
                value='{"name": "Vaishali", "age": 20}'
            )

            required_fields_input = gr.Textbox(
                label="Required Fields (comma-separated)",
                value="name, age, email"
            )

        with gr.Column(scale=1):
            schema_input = gr.Code(
                language="json",
                label="JSON Schema (optional)",
                lines=10,
```

3. Case Conversion: Convert between case styles
   - Input: JSON
   - Output: Converted JSON in target case

```python
# ============= TAB 3: Case Conversion =============
with gr.TabItem("🎨 Case Conversion"):
    gr.Markdown("### 🔄 Convert JSON Key Cases")

    with gr.Row():
        with gr.Column(scale=1):
            json_for_case = gr.Code(
                language="json",
                label="Input JSON",
                lines=10,
                value='{"firstName": "Vaishali", "lastName": "David"}'
            )

            target_case = gr.Radio(
                ["snake_case", "camelCase", "kebab-case", "PascalCase"],
                value="camelCase",
                label="Target Case Style"
            )

            case_convert_btn = gr.Button("🔄 Convert", size="lg", variant="primary")

        with gr.Column(scale=1):
            case_output = gr.Code(
                language="json",
```

4. Merge JSON: Combine multiple objects

- Input: Three JSON objects
- Output: Merged result

```python
# ============= TAB 4: Merging =============
with gr.TabItem("🔗 Merge JSON"):
    gr.Markdown("### 🔄 Merge Multiple JSON Objects")

    with gr.Row():
        with gr.Column(scale=1):
            json1 = gr.Code(
                language="json",
                label="JSON #1",
                lines=8,
                value='{"name": "Vaishali"}'
            )
            json2 = gr.Code(
                language="json",
                label="JSON #2",
                lines=8,
                value='{"age": 20}'
            )
            json3 = gr.Code(
                language="json",
                label="JSON #3 (optional)",
                lines=8,
                value='{"email": "Vaishali@gmail.com"}'
            )
```

5. Flatten/Unflatten: Transform nesting
   - Input: JSON
   - Output: Transformed structure

```python
# ============= TAB 5: Flattening =============
with gr.TabItem("📄 Flatten/Unflatten"):
    with gr.Tabs():
        with gr.TabItem("Flatten"):
            gr.Markdown("### 📦 Flatten Nested JSON")

            with gr.Row():
                flatten_input = gr.Code(
                    language="json",
                    label="Nested JSON",
                    lines=10,
                    value='{"user": {"name": "Vaishali", "age": 20}}'
                )
                flatten_btn = gr.Button("Flatten", size="lg", variant="primary")

            flatten_out = gr.Code(
                language="json",
                label="Flattened JSON",
                lines=10
            )
            flatten_status = gr.Markdown()

            flatten_btn.click(
                flatten_json_func,
```

6. History: Track transformations
   - Download: Export history report
   - Clear: Reset history

```python
# ============= TAB 6: History =============
with gr.TabItem("📜 History"):
    gr.Markdown("### 📊 Transformation History")

    with gr.Row():
        history_btn = gr.Button("📥 Download History", size="lg", variant="secondary")
        clear_history_btn = gr.Button("🗑️ Clear History", size="lg", variant="stop")

    history_output = gr.Textbox(
        label="History Report",
        lines=15
    )

    history_status = gr.Markdown()

    history_btn.click(
        download_history,
        outputs=[history_output]
    )

    clear_history_btn.click(
        clear_history,
        outputs=[history_status]
    )
```

7. Guide: User documentation
   - Examples, tips, and troubleshooting

```
# ============ TAB 7: Guide ============
with gr.TabItem("📖 Guide"):
    gr.Markdown("""
    # 📖 JSON Refiner - Complete Guide

    ## Input Format
    Use simple **Key: Value** format:
    ```
    name: Vaishali David
    age: 20
    is_active: true
    balance: 1500.50
    ```

    ## Type Inference
    **Automatic type detection:**
    - `true`, `false`, `yes`, `no` → Boolean
    - `null`, `none`, `undefined` → Null
    - `123`, `456` → Integer
    - `3.14`, `2.5` → Float
    - `[1, 2, 3]` → Array (if valid JSON)
    - `{"key": "value"}` → Object (if valid JSON)
    - Everything else → String
```

UI/UX Features

Professional interface design:

- Dark theme with gradient background
- Color-coded status messages (✅ success, ❌ error)
- Syntax-highlighted code blocks
- Real-time processing with instant feedback
- Detailed error reporting with line numbers
- Responsive layout for various screen sizes
- Intuitive button-based navigation

MILESTONE 5: Testing & Deployment

Activity 5.1: Google Colab Deployment

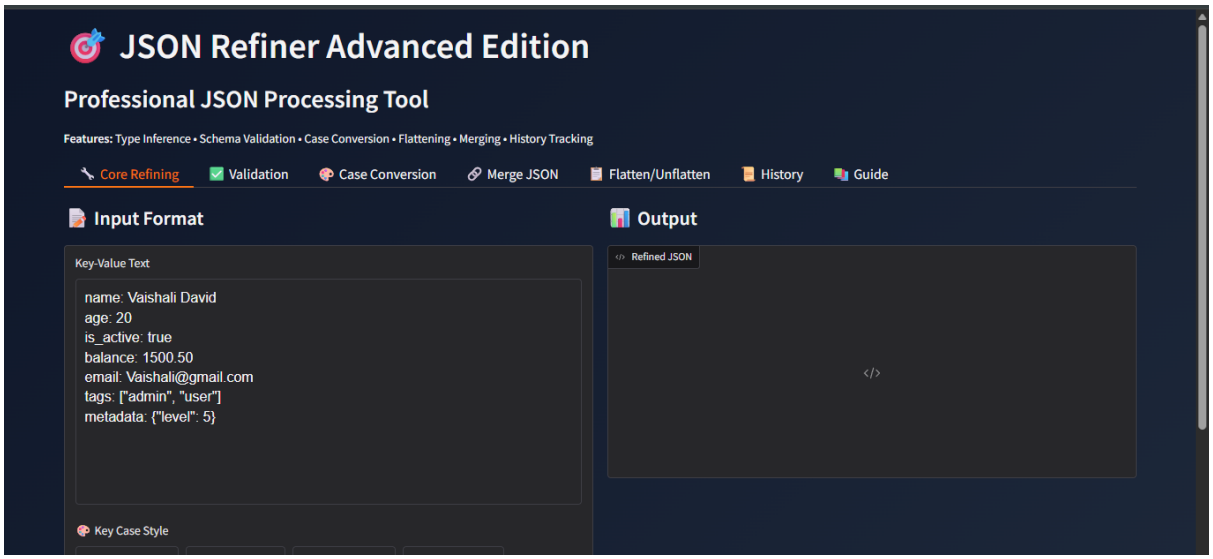
```
print("\n✅ JSON Refiner Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Activity 5.2: Functional Testing

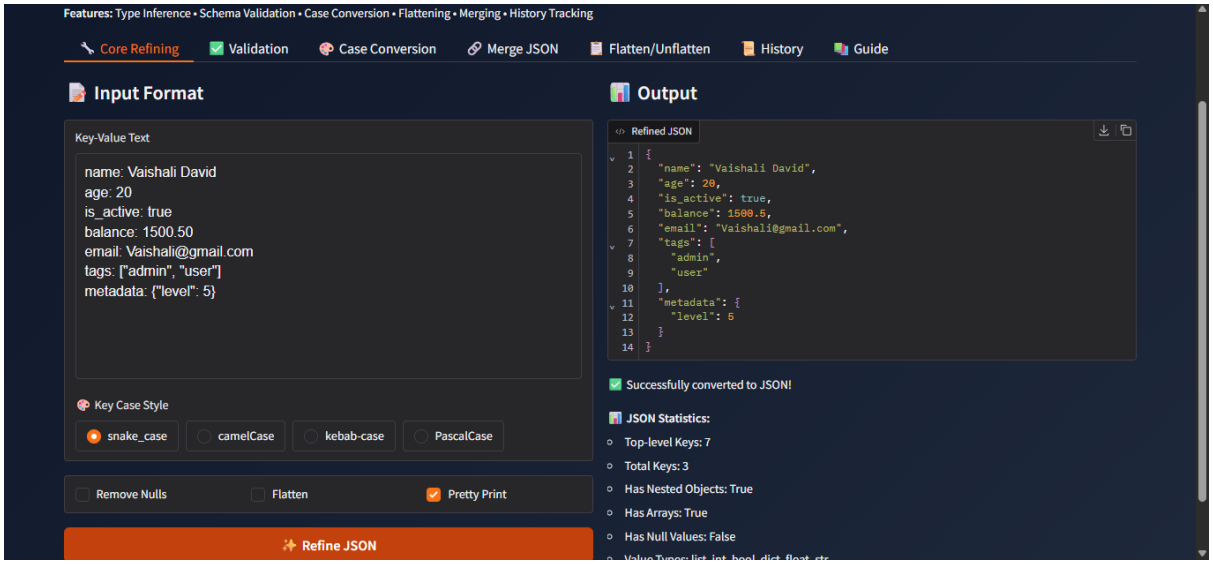
Test Case 1: Basic Key-Value Conversion

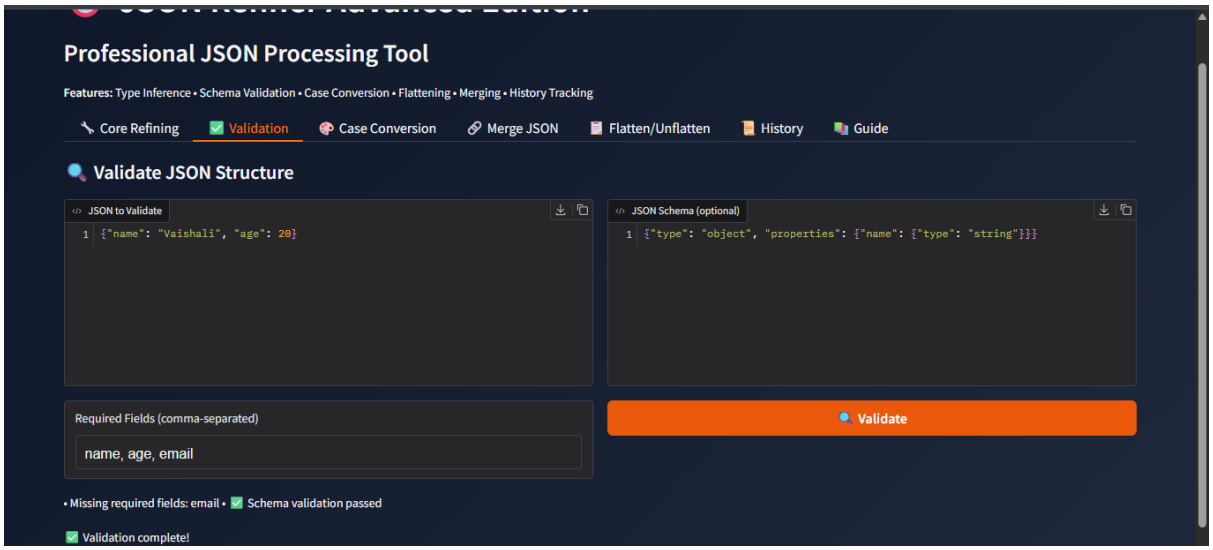
Input:
name: Vaishali Doe
age: 20
active: true
balance: 1500.50

input:



Out put :





JSON Refiner Advanced Edition

Professional JSON Processing Tool

Features: Type Inference • Schema Validation • Case Conversion • Flattening • Merging • History Tracking

Core Refining | Validation | Case Conversion | Merge JSON | Flatten/Unflatten | History | Guide

Convert JSON Key Cases

Input JSON

```
{"firstName": "Vaishali", "lastName": "David"}
```

Converted JSON

```
{
  "first_name": "Vaishali",
  "last_name": "David"
}
```

Target Case Style

snake_case | camelCase | kebab-case | PascalCase

Converted to snake_case!

Convert

JSON Refiner Advanced Edition

Professional JSON Processing Tool

Features: Type Inference • Schema Validation • Case Conversion • Flattening • Merging • History Tracking

Core Refining | Validation | Case Conversion | Merge JSON | Flatten/Unflatten | History | Guide

Merge Multiple JSON Objects

JSON #1

```
{"name": "Vaishali"}
```

JSON #2

```
{"age": 20}
```

Merged JSON

```
{
  "name": "Vaishali",
  "age": 20,
  "email": "Vaishali@gmail.com"
}
```

Merged 3 JSON objects!

Professional JSON Processing Tool

Features: Type Inference • Schema Validation • Case Conversion • Flattening • Merging • History Tracking

Core Refining | Validation | Case Conversion | Merge JSON | Flatten/Unflatten | History | Guide

Flatten | Unflatten

Flatten Nested JSON

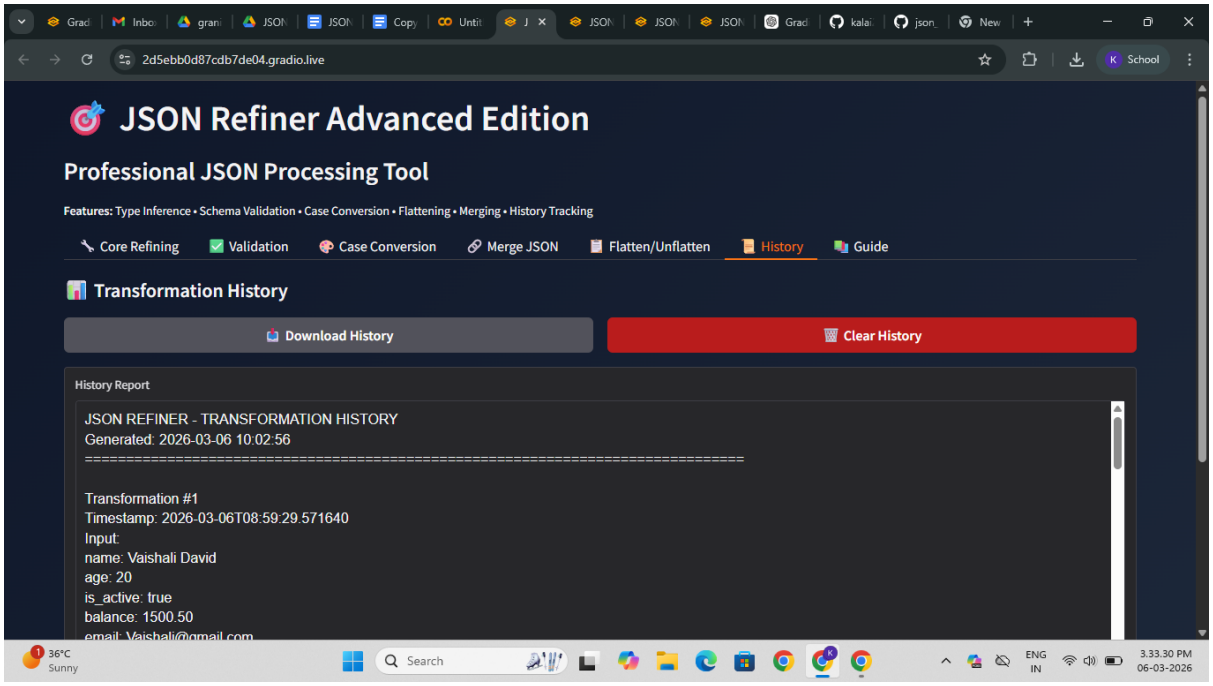
Nested JSON

```
{"user": {"name": "Vaishali", "age": 20}}
```

Flatten

Flattened JSON

```
{
  "user.name": "Vaishali",
  "user.age": 20
}
```



Conclusion

JSON Refiner Advanced Edition successfully delivers an intelligent JSON processing assistant that combines intuitive data transformation with powerful Python processing capabilities. By leveraging Gradio's flexible interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable JSON processing accessible to data analysts, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted data transformation, making professional JSON processing accessible while maintaining high data quality standards, comprehensive validation, and production-ready code output. With automatic type inference, four case conversion standards, advanced transformation operations, comprehensive validation capabilities, and detailed history tracking,

JSON Refiner Advanced empowers users to process complex JSON data efficiently and accurately.

Future Enhancements:

The modular architecture allows for easy extension with additional features such as custom transformation rules, regex-based data extraction, integration with popular data tools (Pandas, NumPy), batch processing capabilities, API endpoint generation, custom validation rule creation, and template-based transformations. The system can be extended to support additional case styles, implement machine learning-based type inference, add data anonymization capabilities, and integrate with cloud storage services for large-scale data processing.

With continued development, JSON Refiner Advanced can evolve into a comprehensive data processing platform supporting diverse data formats while maintaining its focus on simplicity, accuracy, and user experience.